

Ejercicio 10: Puntos Maximales en el Plano (Enfoque Voraz)

Enunciado

Se dice que un punto p del plano euclídeo domina a otro q cuando las coordenadas de p son mayores o iguales que las de q ($p_x \geq q_x$ y $p_y \geq q_y$) y además $p \neq q$. Se dice que un punto p es maximal en un conjunto de puntos C si no existe ningún punto $q \in C$ que domine a p .

Diseñar un algoritmo que, dado un conjunto C de puntos en el plano, calcule el conjunto de todos los puntos maximales de C utilizando una estrategia voraz.

1. Modelado e Idea Voraz

Una solución directa (fuerza bruta) compararía cada punto con todos los demás para verificar si es dominado por alguno, lo que requeriría un tiempo $O(n^2)$.

Para diseñar un algoritmo **voraz y óptimo**, podemos aprovechar la estructura geométrica del problema mediante una **ordenación previa**:

1. **Ordenación inicial**: Ordenamos todos los puntos de C en orden decreciente según su coordenada X . Si dos puntos coinciden en su coordenada X , los ordenamos en orden decreciente respecto a su coordenada Y .
2. **Estrategia Voraz (Barrido / Sweep-line)**: Al estar ordenados de derecha a izquierda (de mayor a menor X):
 - El primer punto de la lista ordenada siempre será un **punto maximal**, ya que ningún otro punto en el conjunto puede tener una coordenada X estrictamente mayor.
 - Llevamos un registro de la mayor coordenada Y encontrada hasta el momento (max_y).
 - Para cada uno de los siguientes puntos en la lista, como sabemos que su coordenada X es menor o igual que la de todos los procesados anteriormente, **solo podrá ser maximal si su coordenada Y es estrictamente mayor que la máxima Y vista hasta ahora** (max_y).
 - Si su coordenada Y es menor o igual que max_y , significa que existe un punto situado "arriba y a la derecha" que lo domina, por lo que lo descartamos (elección voraz).

2. Algoritmo

Pseudocódigo

Entrada: C (Conjunto de n puntos en el plano)

```

// Paso 1: Ordenación voraz
Ordenar C de forma decreciente por la coordenada X.
En caso de empate en X, ordenar de forma decreciente por Y.

// Paso 2: Filtrado voraz
Maximales = [] (Conjunto vacío de salida)
max_y = -infinito

Para cada punto p en C (ya ordenado):
    Si (p.y > max_y):
        Añadir p a Maximales
        max_y = p.y // Actualizar el umbral de dominación

Retornar Maximales

```

3. Demostración y Complejidad

Justificación de Optimalidad (Propiedad Voraz)

- **Correctitud:** Al recorrer los puntos de mayor a menor coordenada X , garantizamos que para cualquier punto actual p_i , todos los puntos procesados anteriormente $p_1, \dots, p_{i-1}$ cumplen que $p_{j,x} \geq p_{i,x}$. Por lo tanto, el único factor que puede hacer que p_i sea dominado por los anteriores es su coordenada Y . Al compararlo con max_y (la mayor Y de los puntos a su derecha), sabemos con total certeza si está dominado o no. Los puntos que procesaremos después de p_i tendrán una X menor o igual, por lo que nunca podrán dominar a p_i .

Análisis de Coste

- **Temporal:**
 - **Ordenación:** $O(n \log n)$ utilizando algoritmos estándar como *MergeSort* o *QuickSort*.
 - **Bucle de decisión (Fase Voraz):** $O(n)$, ya que solo realizamos una pasada lineal sobre los puntos ordenados comparando la coordenada Y con una variable.
 - **Coste total:** $O(n \log n)$. Esto es óptimo para este problema bajo el modelo de árbol de decisión algebraico.
- **Espacial:** $O(n)$ en el peor de los casos para almacenar el conjunto de salida (cuando todos los puntos son maximales).